

Agmarknet Web Scraper — Full Implementation Report

What this system does, how it was built, every issue hit along the way and how each was resolved, and where it stands today. Covers the full history from the original build through the site-break/VM-deployment work done 2026-08-11.

1. What this is and why it exists

Farm Vaidya's voice bot answers "what's the price of X in Y" calls in real time. That answer has to come from somewhere real — India's government mandi (market) price portal, **agmarknet.gov.in**. The portal has no public API; the only way to get its data is to drive the actual website like a person would. That's what this scraper does, on a schedule, continuously, so the bot always has a recent price to give a caller without making them wait for a live page load mid-call.

```
agmarknet.gov.in → agmarknet_scraper.py → PostgreSQL (fact_market_prices)
    (headless
      Playwright
      + captcha OCR)
                                     |
                                     |
                                     price_lookup.py (fuzzy match,
                                     live fallback, LLM tool)
                                     |
                                     Bot.py registers get_price /
                                     get_price_all_markets
                                     |
                                     caller hears an answer
```

2. Original implementation

2.1 Driving the site itself

The core of the scraper (`bot_processors/pricing/agmarknet_scraper.py`) uses **Playwright** to run a real, headless Chromium browser against Agmarknet's "Daily Price and Arrival Report" form — opening it, picking the State/Commodity Group/Commodity dropdowns, clicking "Go," and reading the results table. Nothing about what it scrapes is hand-picked: every state/UT (the site's own "All States/UTs" option) and every commodity (harvested from the site's own dropdown, `harvest_commodity_reference()`) is walked dynamically. Two entry points:

- `scrape_all()` — the bulk job. One query per commodity (not per state × commodity), covering every state at once via "All States/UTs" — roughly 585 unique commodities per full run.
- `scrape_single()` — a one-off, single-commodity/single-state fetch, used mid-call when a caller asks about something not yet cached.

2.2 Captcha

Agmarknet started requiring a captcha on every form submission on **2026-07-31** — the very same day this project's git history begins, so captcha-handling has been part of the scraper from its first working version. Solved

with **EasyOCR** (a PyTorch-backed OCR model) rather than a third-party captcha-solving service, since the image is a clean, low-noise 6-character alphanumeric render. Ambiguous glyphs (l/l/1, 0/O/9) mean a single OCR pass is wrong more often than right, so submission retries against a freshly refreshed captcha image (up to 12 attempts) rather than trusting one guess.

2.3 Reference data, not hardcoding

Commodity names and state/UT names are never hand-maintained lists — `harvest_commodity_reference()` / `harvest_state_reference()` walk the site's own dropdown options once and persist them (`agmarknet_commodities.json` / `agmarknet_states.json`). Everything this bot can answer about comes from what the site itself currently tracks.

2.4 Storage: JSON cache → PostgreSQL

The scraper originally wrote to a single JSON file (`last_known_prices.json`) that got overwritten on every scrape — a snapshot, not history. On **10 August 2026** this was migrated to a real `fact_market_prices` table in PostgreSQL (`db/schema.sql`), with a `UNIQUE (state, district, crop_keyword, arrival_date)` constraint and `ON CONFLICT ... DO UPDATE`, so a re-scrape of the same day updates that day's row while a genuinely new day's data accumulates as real history instead of being discarded. The JSON file is still written alongside it (other tools — `enrich_prices_with_geo.py`, `export_prices_xlsx.py` — read it), but Postgres is the bot's actual source of truth.

2.5 Live fallback

When a caller asks about a commodity/district combination the cache doesn't have (most likely a race with the background job's own refresh cycle), `price_lookup.py` triggers a one-time `scrape_single()` call mid-call rather than telling the caller "no data" — the result is also persisted, so the next caller asking the same thing gets the cached fast path.

3. Automation

`bot_processors/pricing/scraper_daemon.py` is what makes this run unattended: an infinite loop that scrapes today's data, then sleeps 8 hours (3×/day) and repeats. Originally scheduled via Windows Task Scheduler, which kept getting silently disabled — most likely a third-party antivirus product's heuristic flagging a scheduled task that launches a headless browser as malware persistence. Replaced with a plain long-running process, and later (see §5) a proper `systemd` service on a dedicated VM.

4. Issues faced and how each was resolved

Organized roughly chronologically.

4.1 Captcha appears on every submission (31 Jul 2026)

Symptom: every scrape started silently returning 0 rows — the query was being rejected server-side for a missing captcha field. **Fix:** EasyOCR-based solving with retry-on-rejection (§2.2).

4.2 Redundant duplicate scrape pipeline

Symptom: a separate `scrape_nationwide()` / `export_nationwide_xlsx.py` pipeline was running the exact same site queries as `scrape_all()`, just producing a second, unused output. **Fix:** retired — one scraper, one cache file.

4.3 Async checkpoint silently broken by the Postgres migration

Symptom: `scrape_all()`'s per-commodity checkpoint (meant to survive a mid-run crash without losing hours of progress) called the now-async `save_scraped()` without `await` — created a coroutine and discarded it, caught by a `pyright` / `pyflakes` sweep, not by any runtime error. **Fix:** split into a sync `_merge_into_last_known_json()` helper the sync scrape loop calls directly.

4.4 Site restructure broke navigation entirely (~10 Aug 2026)

Symptom: every scheduled scrape had been failing silently for days — `last_known_prices.json` was stuck 4 days stale before anyone noticed. The daemon's log showed every run dying the same way:

```
playwright._impl._errors.TimeoutError: Page.click: Timeout 30000ms exceeded.  
Call log:  
  - waiting for locator("text=Price & Arrival Reports")
```

Agmarknet had removed the homepage nav link the scraper clicked through to reach the form. **Fix:** `_open_daily_price_form()` now navigates straight to the form's own stable URL instead of clicking through the homepage.

4.5 Two dormant bugs in the standalone manual-run entry point

Found while getting `python -m bot_processors.pricing.agmarknet_scraper` working again — both predated the site restructure, just never triggered because nobody had run this entry point recently: - **.env loaded too late:** `db_pool.py` reads `DATABASE_URL` at *import* time; `__main__`'s `load_dotenv()` ran after that import had already executed, so `DATABASE_URL` came back empty. Fixed by moving `load_dotenv()` to the top of the file. - **Playwright's sync API called inside a running asyncio loop:** `scrape_all()` was called directly inside `asyncio.run(...)`, which Playwright's sync API refuses outright. Fixed with `asyncio.to_thread(scrape_all, ...)`.

4.6 No way to scrape a specific past date

Symptom: the rendered form has no date field at all — every query silently hardcodes `from_date` / `to_date` to today, with nothing in the UI to change that. **Investigation:** inspecting the actual network request the "Go" button fires (not the visible form) found it carries explicit `from_date` / `to_date` fields the backend honors for genuinely different past dates — confirmed by fetching real 09-08-2026 rows while the live site only showed the 11th. **Fix:** `_install_date_override()` uses Playwright's `page.route()` to intercept and rewrite that request before it's sent. `scrape_all()` gained a `target_date` parameter and the CLI a `--date=YYYY-MM-DD` flag. No-ops when the target date is today, so normal runs are unaffected. **Caveat:** only verified a few days back — how far the backend's history actually goes is untested.

4.7 The site itself is flaky

Symptom: even with the nav fixed, individual scrape attempts regularly died partway through — transient overlay-click failures, captcha rejections, occasional full page-load timeouts, sometimes cascading into the whole run aborting. **Fix:** `scraper_daemon.py` now retries every scrape (today's included) up to 4 times with `resume=True` on retries after the first, so a late attempt only re-queries commodities an earlier attempt didn't reach. Also added an automatic gap-check: each cycle, one cheap `SELECT DISTINCT arrival_date` query checks the last 3 days in `fact_market_prices`, and backfills any gap found — a permanent safety net against a repeat of §4.4 (days of silent failure going unnoticed).

4.8 Manual backfill of the outage window (8th–11th Aug)

Before the daemon's own auto-backfill existed, the missing days were backfilled by hand with a throwaway retry script — superseded once the daemon gained the same logic natively (§4.7).

5. Moving to a dedicated VM

The scraper now also runs as an always-on service on a separate Azure VM (`sethu-admin1` , Ubuntu 24.04, Central India), rather than depending on a dev laptop staying powered on.

5.1 Prior workload on the VM

The VM wasn't empty — it was running a live `Sethu_Dashboard` stack (`nginx` , a Node backend, two sync services). Confirmed dead/replaced with the project owner before removing it: services stopped and disabled, unit files and `nginx` config deleted, project directory removed.

5.2 Code deployment

Repo is private — a dedicated **read-only SSH deploy key** (scoped to just this one repository, pull-only) was used instead of sharing a personal token.

5.3 Dependency install — GPU build caught before it mattered

`requirements-scraper.txt` — a lean dependency list (just what this subsystem imports) instead of the full ~205-line voice-bot requirements file. Caught during install: `pip` resolved `easyocr`'s unpinned `torch` dependency to the default **GPU/CUDA** build — this VM has no GPU, so that pulled ~3.8GB of unused `nvidia-cu*` / `cuda-toolkit` packages for nothing. Reinstalled from PyTorch's CPU-only wheel index instead — functionally identical, venv went from 5.3GB to 1.5GB.

5.4 Playwright cache landed in the wrong place

First `playwright install --with-deps chromium` was run under `sudo`, which downloaded the browser into **root's** cache directory — the service runs as a non-root user, so it would never have found it. Re-ran without `sudo` (system-level deps were already installed) to land it correctly, removed root's stray copy.

5.5 Database connectivity

`DATABASE_URL` had always pointed at `127.0.0.1:5432` — Postgres on the dev Windows PC. Meaningless from a separate VM. Connected the two machines with **Tailscale** (private mesh VPN, no port-forwarding, no public exposure) rather than exposing Postgres to the internet or migrating it to a managed cloud DB (bigger, deferred

task): - `pg_hba.conf` on the Windows side got one new line, scoped tightly to just the VM's specific IP and the app's own database/user (not a broad `all / all` rule like the localhost entries). - A new Windows Firewall rule opens port 5432, but **only** to Tailscale's own address range (`100.64.0.0/10`) — never the public internet. - Config reload (`pg_ctl reload`), not a full service restart, so nothing already connected got dropped.

5.6 systemd service

`agmarknet-scraper.service` replaces the Windows-only `start_scraper_daemon.vbs`. Enabled (survives reboot), `Restart=on-failure`.

5.7 Verification before calling it done

Each layer tested independently, then the whole chain together: real scrape → real captcha OCR → real write to Postgres over the Tailscale tunnel → confirmed by reading the rows back with an independent query. Only after that did the systemd service go live.

6. A downstream ordering confusion (not a scraper bug)

Browsing `fact_market_prices` directly in pgAdmin's default "View/Edit Data" grid showed rows in no particular date order — expected Postgres behavior (a table has no inherent row order without `ORDER BY`), not scrambled data. Added `fact_market_prices_ordered`, a view wrapping the table with `ORDER BY arrival_date, state, district, commodity`, purely so manual browsing comes back sorted without typing `ORDER BY` every time. Not read by any application code.

7. Current status (as of this report)

Date	Rows in <code>fact_market_prices</code>
01–07 Aug 2026	Present (pre-dates this session's work)
08 Aug 2026	3,401 (backfilled)
09–10 Aug 2026	Backfilling in progress
11 Aug 2026	3,406 (first automated VM cycle, completed)

Going forward, the VM's `scraper_daemon.py` handles both today's data and gap-backfilling for the last 3 days automatically, every 8-hour cycle — no manual runs required for the steady state.

8. Tools and technologies used

Tool	Role
Playwright (headless Chromium)	Drives the real site — the core of the pipeline
EasyOCR + PyTorch (CPU)	Solves the site's captcha locally
Python <code>asyncio</code> loop	The daemon's "run, sleep 8h, repeat" logic
systemd	Supervises that loop on the VM — auto-restart, auto-start on boot
PostgreSQL + <code>asyncpg</code>	Where every scraped price is actually stored
Tailscale	Private VPN connecting the VM to the database

Tool	Role
Azure VM (Ubuntu 24.04)	Where the automated copy runs 24/7
Git + GitHub (read-only deploy key)	How code reaches the VM
loguru, openpyxl, python-dotenv	Logging, xlsx export, config loading

9. Known limitations

- **Database availability is tied to the dev Windows PC.** Tailscale removes the need for port-forwarding, not the underlying dependency — if that PC is off, the VM's scraper can run but has nowhere to save results. A managed cloud Postgres would remove this; deferred as a separate, larger task.
- **Date-override backfill (§4.6) is only verified a few days back** — don't assume it works for an arbitrarily old date without testing that specific date first.
- **The scraper's own data is now correct going forward, but the voice bot's *consumption* of it had a separate, unrelated bug** (the LLM briefly read an internal instruction string aloud to a caller) — documented separately, not part of this scraping pipeline itself.